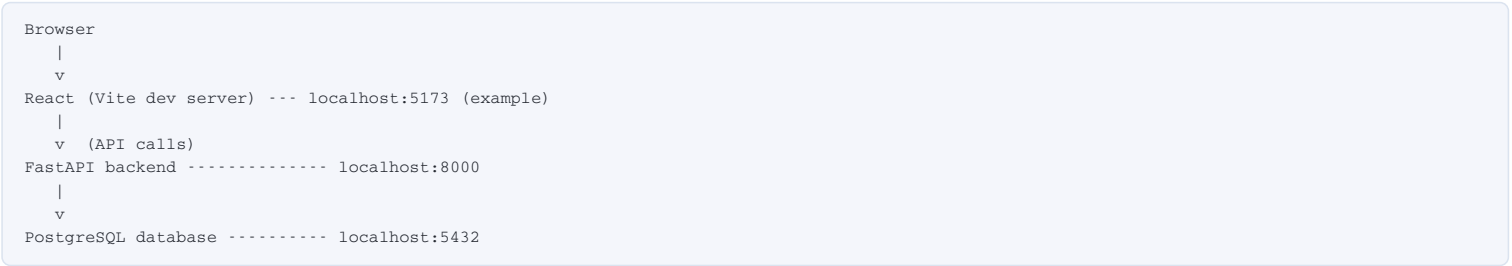


AI Platform — Docker Networking Report

How the frontend, backend and database talked before Docker, and how they talk now — ports, URLs and machine-to-container communication explained simply.

1. Before Docker — everything on one laptop

Before Docker, all three parts of the app ran directly on your laptop as normal programs. They talked to each other using `localhost`, because "localhost" simply means **"this same machine."** Since everything was on the same machine, that worked fine.



Backend's database connection looked like:

```
postgresql+asyncpg://ai_platform_user:abdullah@localhost:5432/ai_platform
```

This worked only because the backend process and the database were both running directly on your laptop, not inside any container.

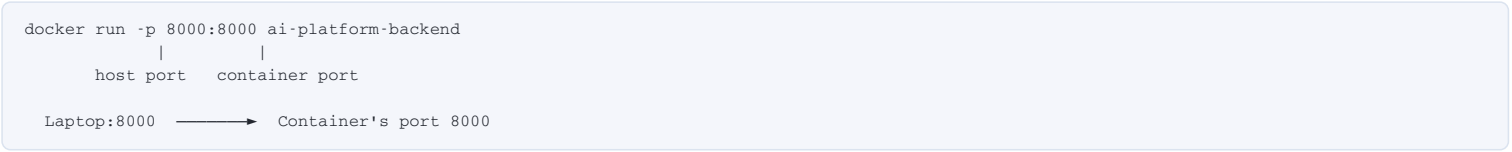
2. The problem Docker introduces: "localhost" changes meaning

Key lesson: Once a program runs *inside* a container, `localhost` no longer means "my laptop." It means **"this container only."** A container cannot reach another container through `localhost`.

This is why, when the backend was first put in a container and still pointed at `localhost:5432`, it could not find the database anymore — the database was in a different container, i.e. a different "localhost."

3. How your laptop talks to Docker

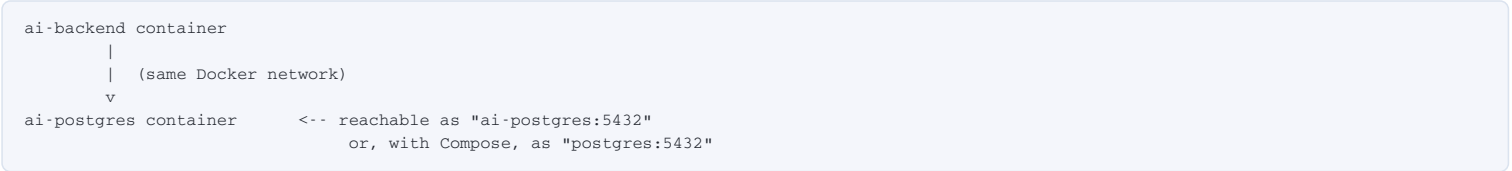
Docker runs on your laptop as a background engine (Docker Desktop / Docker daemon). Containers live inside this engine, somewhat like small isolated mini-computers. Your laptop (the "host") only sees a container if a **port is published** to it.



Without `-p HOST_PORT:CONTAINER_PORT`, the container is invisible to your laptop's browser or terminal — you simply couldn't reach it.

4. How containers talk to each other

Containers reach each other through a **Docker network**, not through the laptop. When two containers join the same network, Docker gives them an internal DNS so they can find each other by **name** instead of IP address (IPs can change, names don't).

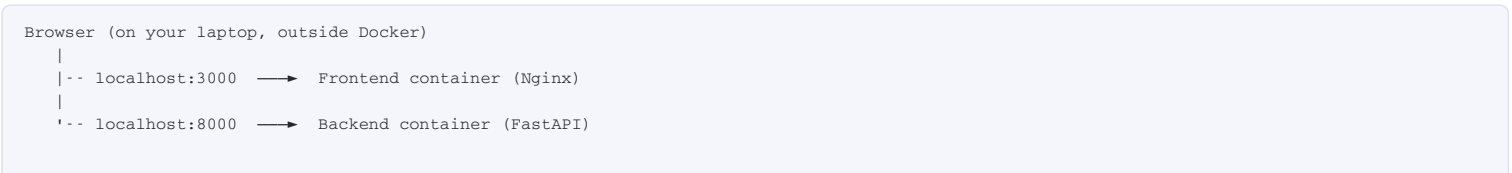


So the backend's database URL changed from `localhost` to the container/service name:

```
postgresql+asyncpg://ai_platform_user:abdullah@postgres:5432/ai_platform
```

5. How the browser talks to everything (important distinction)

The browser runs **outside** Docker entirely, on your laptop. It has no idea Docker or its internal network exists. So the browser can only ever use `localhost:PUBLISHED_PORT` — never a container name like `backend` or `postgres`.



Inside Docker (invisible to the browser):
Backend container → postgres:5432 → PostgreSQL container

That's why the React frontend still calls the backend using `http://localhost:8000` in the browser, even though, internally, Docker also knows the backend by its service name.

6. Full picture — before vs. after

	Before Docker	After Docker (Compose)
Where things run	Directly on laptop	Inside 3 containers
Frontend	Vite dev server	Nginx serving built React files
Backend reaches DB via	<code>localhost:5432</code>	<code>postgres:5432</code> (Docker network name)
Browser reaches frontend via	<code>localhost:5173</code>	<code>localhost:3000</code>
Browser reaches backend via	<code>localhost:8000</code>	<code>localhost:8000</code> (unchanged, just published from a container now)
Data storage	Local Postgres install	Docker volume <code>ai-platform-data</code> , separate from the container

7. Ports used in this project

Service	Host port (laptop)	Container port
Frontend (Nginx)	3000	80
Backend (FastAPI)	8000	8000
PostgreSQL	5433	5432

Postgres uses 5433 on the host (not 5432) because your laptop already had its own separate Postgres installed on 5432, so 5433 avoids clashing with it.

8. Project overview (current state)

The AI Platform is now a 3-service, Dockerized full-stack app, all started together with one command:

```
docker compose up -d
```



Alembic still creates and updates the database tables — Docker only packages the environment, it doesn't replace migrations. Docker also does **not** automatically give you backups, replication, or production hosting — those are separate concerns you'd still need to set up.

9. Moving the project to a different machine

Because everything needed to run the app is described in files inside the project (not installed by hand), setup on a new machine is short:

1. Install Docker + Docker Compose on the new machine.
2. Copy/clone the project (it already contains the `Dockerfile` s, `compose.yaml` , `requirements.txt` , and Alembic migrations).
3. Run `docker compose up -d --build` — this builds the images and starts all 3 containers.

Migrations used to be a manual 4th step here — see section 13, this is now done automatically.

You do **not** need to manually install Python, Node.js, or PostgreSQL on the new machine — Docker brings all of that inside the containers, identically every time. This is the main benefit of Dockerizing: the same setup works the same way on Linux or Windows.

Note: the actual database *data* lives in the Docker volume, which is local to whichever machine it was created on. Moving to a new machine gives you a fresh, empty database unless you also copy/restore the volume separately.

10. Docker concepts used — quick glossary

Concept	What it means, simply

Image	A packaged blueprint for a program and everything it needs (e.g. <code>python:3.10-slim</code> , <code>postgres:18</code>). Not running by itself.
Container	A running copy of an image. Can be started, stopped, deleted — it's disposable.
Dockerfile	A recipe/text file describing how to build an image step by step (what base to use, what to install, what to run).
Volume	Storage that lives outside the container, so data (like the database) survives even if the container is deleted.
Network	Lets containers find and talk to each other by name instead of IP address.
Port mapping (<code>-p</code>)	Connects a port on your laptop to a port inside a container, so the laptop/browser can reach it.
EXPOSE	Just documents which port the app uses inside the container — it does not open anything to the laptop by itself.
Image layers	Images are built in steps (layers); unchanged steps are reused on rebuild, making builds faster.
Multi-stage build	A Dockerfile with two phases: one to build the app (with heavy tools like Node), one slim final image to run it (e.g. just Nginx). Keeps the final image small.
Docker Compose	A file (<code>compose.yaml</code>) that describes all services (frontend, backend, database) together, so one command starts everything, wires up the network, and reuses the volume.
Service (Compose)	A definition of what should run (e.g. "backend"); Compose turns each service into a container.

11. Common confusion — "did the code change?"

Short answer: **no source code was changed**. Everything was done from the outside, through the `Dockerfile` , command-line flags (`docker run -p ...`), and `compose.yaml` . None of these touch your React or FastAPI source files.

Why port 3000 now, instead of 5173?

These are not "the same port renamed" — they're two different serving setups:

- **Before:** `npm run dev` started Vite's dev server, which defaults to port `5173` .
- **Now:** the frontend is *built* into static files (`npm run build`) and served by **Nginx**, which defaults to port `80` inside the container.

`3000` is simply the number chosen on the host side when writing `-p 3000:80` — it could have been `4000:80` or anything else. It has no real connection to `5173` ; it's a brand-new server (Nginx) with its own port, exposed however you chose.

The one thing that did change — but not in a code file

The backend's database address had to become `postgres:5432` instead of `localhost:5432` to work inside Docker. But this was passed in from **outside** the code, as an environment variable:

- `docker run -e DATABASE_URL="..."` , or
- `environment:` inside `compose.yaml`

This overrides the value at runtime. The actual `.env` file used for local/non-Docker runs was never edited.

Can the project still be run the old way, without Docker?

Yes — as long as `.env` still says `localhost` and not `postgres` . If so:

```
npm run dev                # frontend on :5173, same as before
uvicorn app.main:app        # backend on :8000, same as before
```

This works exactly like before, using the local PostgreSQL install, completely independent of Docker.

Docker and the old manual setup can coexist — they just shouldn't use the same ports at the same time (e.g. don't run Docker's Postgres on 5432 while the local one is also on 5432). This is exactly why Docker's Postgres was mapped to host port `5433` instead — to avoid that clash.

12. Daily Docker workflow — rebuilding, stopping, and when it's needed

Code changes don't touch a running image automatically

An **image** is a frozen snapshot (code + dependencies at the time it was built). A **container** is just a running copy of that image. If you edit your code, the existing image and container don't change by themselves.

Image A → Container A (old code, still running)

you edit code, then run:
docker compose up -d --build

Image A' → New Container (new code)
(Image A may still sit unused on disk — it isn't auto-deleted; clean it later with "docker image prune")

Stopping ≠ deleting

`docker compose stop` just pauses the containers — nothing is lost. `docker compose start` (or `up -d` again) brings them right back. You don't need to create/delete containers every day.

Adding a new package (pip/npm)

Adding something to `requirements.txt` or `package.json` does not install itself into a running container. You must rebuild:

```
docker compose up -d --build
```

This bakes the new dependency into a fresh image.

Recommended day-to-day workflow

While actively coding: just run Vite (`localhost:5173`) and FastAPI (`localhost:8000`) normally, outside Docker — fast hot-reload, no rebuild needed for every small change.

Once a feature is ready: run `docker compose up -d --build` to verify it also works fully containerized, then commit/push.

Docker doesn't replace normal local development — it's used to confirm the containerized version still works before shipping.

How this looks in real production

```
Developer → git push → CI/CD → build image → push to registry
                                     |
                                     ▼
                        Production server runs
                        the new container
```

Production setups usually tag images with version numbers (e.g. `backend:1.4.0` , `backend:1.5.0`) instead of only `latest` , so deployments can be tracked and rolled back safely if something breaks.

Five things to remember

1. Code changes don't automatically update an existing image.
2. `docker compose up -d --build` rebuilds the image and recreates only the affected container(s).
3. Old, unused images stick around until you clean them up — no rush.
4. `docker compose stop` only pauses containers; nothing is deleted.
5. Normal (non-Docker) development is still the everyday workflow — Docker is for verifying the full containerized app before committing/deploying.

13. Automatic migrations + backend waiting for the database

Earlier, tables had to be created by hand after startup, with `docker exec ai-backend alembic upgrade head` . That manual step is now gone — the backend does it itself.

How: an entrypoint script

A new file, `backend/entrypoint.sh` , now runs migrations first, then starts the server:

```
#!/bin/sh
set -e
alembic upgrade head
exec uvicorn app.main:app --host 0.0.0.0 --port 8000
```

```
Backend container starts
      |
      ▼
Alembic migrations run
      |
      ▼
FastAPI starts
```

The backend's Dockerfile now runs this script instead of starting Uvicorn directly. If migrations fail, FastAPI never starts — which is intentional, so a broken database schema can't silently serve traffic.

Making sure Postgres is actually ready first

A PostgreSQL **healthcheck** (using `pg_isready`) was added, so Compose knows the database is truly ready to accept connections — not just that the container has started (those aren't the same moment).

```
PostgreSQL starts
      |
      ▼
Healthcheck passes
      |
      ▼
Backend starts → migrations run → FastAPI starts
```

This stops the backend from trying to migrate before Postgres is actually accepting connections.

Compose can now build the backend itself

The backend service in `compose.yaml` previously just pointed at a pre-built image. It now includes:

```
build:
  context: ./backend
```

So `docker compose up -d --build` builds the backend image itself — no separate manual `docker build` command needed anymore.

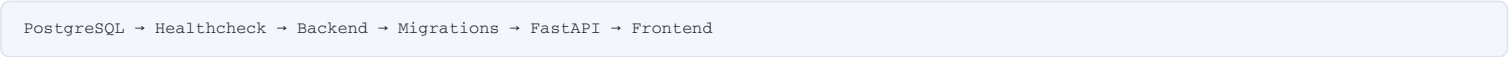
Nothing about the data or the app logic changed

The existing `ai-platform-data` volume was kept as-is — the database was not deleted or recreated, so existing data survives container restarts/rebuilds. Application logic, API behavior, and the migration files themselves were untouched — this was purely a startup/orchestration improvement.

Result: true "clone → up → ready" setup

```
git clone <repository>
cd ai-platform
docker compose up -d --build
```

Compose now handles the entire chain automatically:



Handy commands

Action	Command
Start	<code>docker compose up -d</code>
Start + rebuild (after code/dependency changes)	<code>docker compose up -d --build</code>
Stop (keeps everything, just pauses)	<code>docker compose stop</code>
Resume stopped containers	<code>docker compose start</code>
Check status	<code>docker compose ps</code>
View backend logs	<code>docker compose logs backend</code>